Clase 10 · Taylor con la FPU e interfaz PIO

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · lunes 12 oct 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Proyecto: seno y coseno por series de Taylor (FPU de la Pico 2)

La Pico 2 no tiene instrucciones de hardware para seno o coseno: se calculan por software con sus polinomios de Taylor, en ensamblador.

- **sen(x) = x − x^3/3! + x^5/5! − x^7/7! + …** (exponentes y factoriales **impares**).
- **cos(x) = 1 − x^2/2! + x^4/4! − x^6/6! + …** (**pares**).
- No agregar demasiados términos: basta una aproximación funcional.

> **Recomendación de diseño**
>
> Las divisiones son lentas: **precalcular los factoriales** y multiplicar por el recíproco como constante. En la FPU de la Pico 2 (Cortex-M33) una multiplicación de punto flotante toma **1 ciclo**.
>
> Solo corre en la Pico 2 (por su FPU); en la Pico 1 habría que emular por software y sería muy lento. Es el Proyecto 1; continúa la FPU de las Clases 08 y 09.

2. La PIO (Programmable I/O)

Periférico de la Pico formado por pequeños coprocesadores independientes ("pseudo-FPGA"), las **máquinas de estado**, que manejan pines de E/S en paralelo y a velocidad de hardware.

- Se comunican con el bus por **FIFO** de entrada y salida; soportan **interrupciones y DMA**.

Placa	Máquinas de estado
Pico 1	**8** (2 bloques PIO de 4)
Pico 2	**12** (3 bloques de 4)

Periférico	Hardware de la Pico	Emulado con PIO
UART	460 kbps	"Super-UART" hasta 12 MHz
SPI	7.8 MHz	24–32 MHz
Video	—	VGA, HDMI, DisplayPort (barras de color)

Libera a la CPU de los tiempos estrictos en los pines.

3. Juego de instrucciones del PIO Assembler

Instrucción	Función

PUSH / PULL	Meter o sacar datos de la FIFO
IRQ	Generar o manejar banderas de interrupción
IN / OUT	Pines → registros de corrimiento / registros → pines
SET	Asignar valores directos a pines o registros
WAIT	Detenerse hasta una condición en un pin
MOV	Mover datos entre registros
JMP	Salto condicional o incondicional — **1 ciclo** (en la CPU ARM Harvard cuesta más)
NOP	No hace nada; ajusta tiempos

- Registros por máquina: **X y Y** (32 bits, scratch), **ISR** y **OSR** (corrimiento de entrada/salida) y divisores de reloj.
- **Retardo por instrucción:** cada instrucción dura 1 ciclo y se le anexa un retardo en la misma línea (hasta ≈ 32): un MOV con retardo 9 dura 10 ciclos.

Error común: confundir X y Y con R0–R15 de la CPU ARM.

4. PIO en MicroPython: parpadeo a 1 Hz

```
import rp2
from machine import Pin
import time

@rp2.asm_pio(set_init=rp2.PIO.OUT_LOW)
def encender():
    set(pins, 1)              # pin en 1
    set(x, 31)       [6]      # X = 31, con retardo
    label("delay_high")
    nop()            [29]     # 1 + 29 = 30 ciclos
    jmp(x_dec, "delay_high")  # 1 ciclo
    set(pins, 0)
    set(x, 31)       [6]
    label("delay_low")
    nop()            [29]
    jmp(x_dec, "delay_low")

sm = rp2.StateMachine(1, encender, freq=2000, set_base=Pin(25))
sm.active(1); time.sleep(4); sm.active(0)   # 4 destellos
```

Parte	**Ciclos**
Poner el pin y cargar X	1 + 7 = 8
Bucle: X = 31 → 32 vueltas × (30 + 1)	992
Fase encendida	1000
Fase apagada (simétrica)	1000
Periodo	**2000**

Frecuencia

Reloj del sistema: 125 MHz (Pico 1) y 150 MHz (Pico 2); los divisores se ajustan solos a los 2000 Hz pedidos. 2000 ciclos a 2000 Hz → **1 Hz**. `freq` es el reloj de la máquina, no la frecuencia del parpadeo.

Avisos

- Proyecto de Taylor: fecha ambigua; confirmar. Material PIO prometido. Próxima clase: fórmulas de tiempo y contador de frecuencia.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Taylor. Seno: potencias y factoriales impares; coseno: pares. Precalcular factoriales y multiplicar (1 ciclo en la FPU de la Pico 2). Solo en Pico 2.

PIO. Máquinas de estado independientes con FIFO, IRQ y DMA: Pico 1 = 8, Pico 2 = 12. Emula UART (12 MHz), SPI (24–32 MHz) y video.

Ciclos (examen)

8 de preparación + 32 × 31 del bucle = 1000 por fase; 2000 por periodo; a freq = 2000 Hz → 1 Hz. JMP = 1 ciclo; `nop() [29]` = 30.

Instrucciones	**PUSH, PULL, IRQ, IN, OUT, SET, WAIT, MOV, JMP (+ NOP)**
Registros	X, Y, ISR, OSR
Retardo	`[n]` suma n ciclos a la instrucción